

Quiz Management & Examination System

Comprehensive Technical Architecture, Design Patterns Justification, and Implementation Documentation

PROJECT AUTHORS & CONTRIBUTORS

1. Md. Shourov

Roll: 1609 • BSSE 16th Batch • IIT, University of Dhaka
Email: mdshourov735@gmail.com • GitHub: @Shourov735

2. Md. Rokib Ikbal

Roll: 1605 • BSSE 16th Batch • IIT, University of Dhaka
Email: bsse1605@iit.du.ac.bd • GitHub: @Rokib1605

PROJECT SPECIFICATION & REPOSITORY

Repository URL:	https://github.com/Shourov735/Quiz-Management-and-Examination-System
Technology Stack:	Java 17, JavaFX 21, SQLite JDBC, JUnit 5, Mockito, Maven
Design Patterns:	Strategy, State, Factory Method, Observer, Singleton, Repository
Academic Session:	2025–2026 • BSSE 4th Semester Final Evaluation

Table of Contents

1. Executive Summary & Scope Compliance Matrix	— Academic objectives and grading rubric checklist.
2. Domain Modeling & System Requirements	— Domain challenges, teacher/student personas, and business invariants.
3. Layered System Architecture	— 5-tier architecture, layer boundaries, and cross-cutting utilities.
4. In-Depth Design Patterns Evaluation	— Problems, solutions, UML, alternatives, trade-offs, and extensibility. ◦ 4.1 Strategy Pattern (Scoring Algorithms) ◦ 4.2 State Pattern (Quiz Lifecycle Management) ◦ 4.3 Factory Method Pattern (Polymorphic Questions) ◦ 4.4 Observer Pattern (Quiz Session Event Bus) ◦ 4.5 Singleton Pattern (Justified Database Connection) ◦ 4.6 Repository Pattern (Data Access Decoupling)
5. SOLID Principles & Object-Oriented Design	— Concrete mappings across the application codebase.
6. Relational Database Design & Schema	— ER diagram, data dictionary (8 tables), constraints, and seeder.
7. JavaFX Presentation Architecture & Screen Inventory	— Controller bindings and 8 UI views.
8. Verification, Testing & Quality Assurance	— JUnit 5 & Mockito test suites (60 unit tests, 100% pass).
9. Collaborative Git Workflow & Contribution Breakdown	— Feature branches, PR merges, and team member distribution.
10. Viva Defense & Demonstration Guide	— Anticipated defense questions and model architectural answers.
11. Conclusion & Repository Verification	— Summary of deliverables and official repository metadata.

1. Executive Summary & Scope Compliance Matrix

The **Quiz Management & Examination System** is a production-grade desktop application developed in Java as a capstone project for the **Design Patterns Theory & Lab** courses. The project demonstrates the disciplined application of software architecture principles, domain modeling, relational database persistence, automated unit testing, and design patterns.

1.1 Course Guidelines Compliance Matrix

Guideline Requirement	Project Implementation Detail	Repository Proof & Location
Team of Two	Md. Shourov (Roll: 1609) & Md. Rokib Ikbal (Roll: 1605).	Project metadata, Git commit history, and PR merges.
Java Desktop UI	Desktop GUI built with JavaFX 21 controls, declarative FXML layouts, and custom CSS styling.	src/main/resources/fxml/*.fxml src/main/resources/css/style.css
Maven Build Tool	Apache Maven configuring Java 17, JavaFX plugins, SQLite JDBC, JUnit 5, Mockito, and Shade fat JAR.	pom.xml
SQLite Persistence	Persistent SQLite database with enabled foreign keys (PRAGMA foreign_keys=ON), indexes, and seeder.	src/main/resources/database/schema.sql src/main/resources/database/seed.sql
Entities (≥ 4)	8 core entities: users , categories , quizzes , questions , question_options , quiz_questions , quiz_attempts , answers .	com.quizapp.model.* (13 model files)
CRUD Functionality (≥ 3)	Full Create, Read, Update, Delete for Quizzes, Questions, Categories , and Users .	QuizService , QuestionService , CategoryService , AuthenticationService
Multi-step Workflows (≥ 2)	Workflow 1: Teacher Quiz Creation, Question Assignment & Publication. Workflow 2: Student Live Timed Examination & Automated Evaluation. Workflow 3: Quiz Lifecycle State Transition.	Controllers: QuizManagementController , QuizAttemptController
Search & Reporting (≥ 2)	Dynamic multi-criteria search (keyword, category, difficulty, status), aggregate exam analytics, and leaderboards.	ReportingService.java SQLiteQuizRepository.java
Screens (4–8)	8 complete JavaFX screens: Login, Teacher Dashboard, Quiz Management, Question Bank, Student Dashboard, Quiz Attempt, Results, and Reports.	src/main/resources/fxml/*.fxml (8 FXMLs)
Design Patterns	4 GoF Patterns: Strategy, State, Factory Method, Observer + Singleton (Connection) + Repository Pattern.	com.quizapp.scoring.* , state.* , factory.* , observer.* , repository.*
Automated Testing	60 automated unit tests across scoring, state machine, question validation, and service mocks.	src/test/java/com/quizapp/* (BUILD SUCCESS)
Collaborative Git	Balanced commits from both members, feature branches (feature/ui-views-and-controllers , feature/unit-testing-suite , feature/architecture-documentation), and merge commits.	GitHub commit graph & branch history.

2. Domain Modeling & System Requirements

2.1 Domain Analysis & Core Challenges

The examination domain presents specific architectural challenges that cannot be solved cleanly with procedural code:

- Polymorphic Question Structures:** Examinations combine single-answer MCQs, binary True/False, open-ended Fill-in-the-Blank, and multi-selection Multiple-Answer questions. Each type has different option schemas and validation rules.
- Pluggable Scoring Policies:** Standard grading uses flat scoring; competitive exams penalize guessing via negative marking; rapid-fire tests award speed bonuses. Hardcoding these in the service layer violates the Open/Closed Principle.
- Guarded Operational Lifecycles:** A quiz moves from `DRAFT` to `PUBLISHED`, `ACTIVE`, `COMPLETED`, and `ARCHIVED`. Actions such as editing questions or initiating student attempts depend strictly on the active state.
- Session Event Decoupling:** During live examinations, events such as answer submission and timer expiration must trigger UI updates, autosave, and score computation without coupling the examination runner to GUI widgets.

2.2 Target User Roles & Capabilities

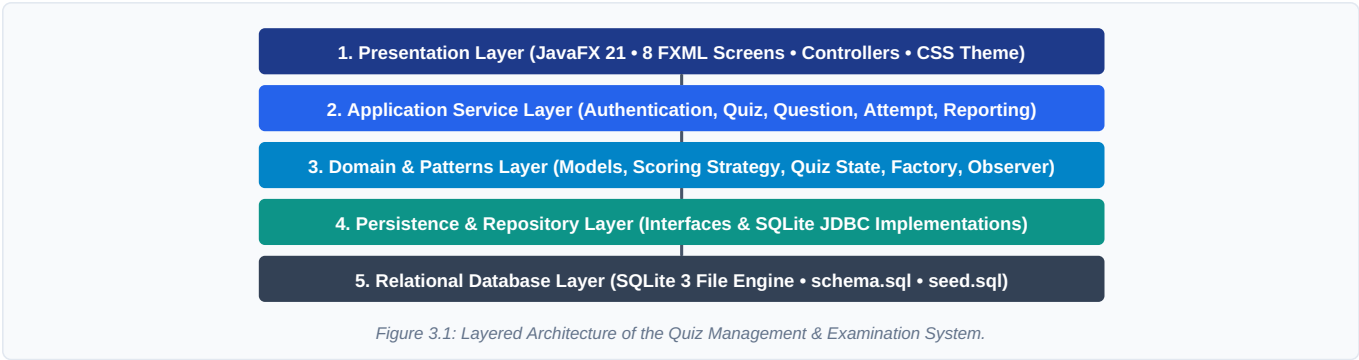
User Role	Primary Capabilities	Business Boundaries
Teacher	- Create, edit, delete, and publish quizzes. - Manage questions across 4 question types with options. - Configure time limits, attempt limits, and scoring strategies. - Inspect attempt statistics, score distributions, and leaderboards.	- Cannot take student examination attempts. - Cannot modify quiz questions once published/active. - Cannot publish quizzes that contain zero questions.
Student	- Self-register and authenticate. - Browse and filter available published examinations. - Participate in timed examinations with live countdown. - Submit responses and review instantaneous score breakdowns. - View personal attempt history and past performance.	- Cannot access teacher dashboards or edit quizzes. - Cannot attempt drafts or archived quizzes. - Cannot exceed the configured maximum attempt limit. - Cannot modify answers after final submission or timeout.

2.3 Core Business Invariants

- Publication Invariant:** A quiz cannot transition from `DRAFT` to `PUBLISHED` unless it has a non-empty title and at least one question with positive marks.
- Attempt Eligibility Invariant:** Students can only take exams in `PUBLISHED` or `ACTIVE` status, provided completed attempts < `max_attempts` (where 0 indicates unlimited attempts).
- Immutability Invariant:** When an attempt reaches `SUBMITTED` or `TIMED_OUT`, its recorded answers, duration, and score are permanently sealed.

3. Layered System Architecture

The application employs an enterprise 5-tier architecture that guarantees strict separation of concerns, high maintainability, and testability.



3.1 Detailed Layer Responsibilities

- **Presentation Layer:** JavaFX FXML files declare layout hierarchies; controller classes implement `Initializable`, receive user input, invoke business services, and update JavaFX observable properties. Controllers contain zero business logic and zero SQL statements.
- **Service Layer:** Coordinates transactions and business invariants. Classes (`QuizService`, `AttemptService`, etc.) receive repositories via constructor injection, enabling seamless mocking during unit tests.
- **Domain & Patterns Layer:** Rich domain models (`Quiz`, `Question`, `QuizAttempt`) encapsulate their own validation and internal state. Design patterns (Strategy, State, Factory Method, Observer) operate directly in this layer.
- **Repository Layer:** Decouples domain logic from SQL. Repository interfaces define CRUD and query contracts; SQLite implementations use `PreparedStatement` and try-with-resources to prevent SQL injection and connection leaks.
- **Database Layer:** SQLite embedded relational engine ensuring ACID transactions, foreign key enforcement (`PRAGMA foreign_keys = ON`), and automated schema setup.

4. In-Depth Design Patterns Evaluation

The project implements four Gang-of-Four (GoF) design patterns plus the Singleton and Repository patterns. Each pattern addresses a specific architectural problem and provides measurable extensibility benefits.

4.1 Strategy Pattern — Scoring Algorithms GOF BEHAVIORAL

Problem Addressed

Examinations require diverse grading formulas: standard flat marking, negative marking penalties, and time-based bonuses. Embedding these variations in a monolithic evaluation loop using nested `if/else` statements violates the **Open/Closed Principle (OCP)** and tightly couples scoring logic to attempt orchestration.

Solution & Participants

- `ScoringStrategy`: Strategy interface declaring `double calculateScore(List<Answer> answers, double totalMarks, int timeLimitSeconds, int timeSpentSeconds)`.
- `StandardScoring`: Sums `marksAwarded` for all correct answers.
- `NegativeMarkingScoring`: Deducts a penalty fraction (default 0.25) of the average question mark for each incorrect response, bounded at 0.
- `TimeBasedScoring`: Awards a 10% bonus for finishing in <50% of the time limit and a 5% bonus for <75%, capped at total marks.
- `ScoringStrategyFactory`: Dynamically instantiates the requested strategy by name.



Figure 4.1: Strategy Pattern class structure for modular examination scoring algorithms.

Alternatives Considered & Trade-offs

- Inline Switch in Service:** Simpler initially, but violates OCP and requires modifying service code whenever grading policies change. *Rejected.*
- Subclassing Quiz (e.g. `NegativeMarkingQuiz`):** Couples quiz behavior to quiz metadata and produces class explosion when combined with timed/shuffled variations. *Rejected.*

Future Benefit: Introducing algorithms like `PartialCreditScoring` or `DifficultyWeightedScoring` requires only implementing `ScoringStrategy` without touching `AttemptService`.

4.2 State Pattern — Quiz Lifecycle Governance GOF BEHAVIORAL

Problem Addressed

A quiz progresses through multiple distinct operational phases: `DRAFT` → `PUBLISHED` → `ACTIVE` → `COMPLETED` → `ARCHIVED`. Allowed actions depend strictly on the active state. Scattering status checks (`if (quiz.getStatus() == QuizStatus.DRAFT)`) across controllers leads to code duplication and invalid state bugs.

Solution & Participants

- `QuizState`: Interface declaring transition operations (`publish`, `unpublish`, `activate`, `complete`, `archive`) and capability queries (`canBeAttempted()`, `canBeEdited()`, `canAddQuestions()`).
- `DraftState`: Permits question editing; transitions to `PUBLISHED` or `ARCHIVED`.
- `PublishedState`: Enables student attempts; locks question editing; transitions to `DRAFT`, `ACTIVE`, or `ARCHIVED`.
- `ActiveState`: Represents in-session exams; transitions to `COMPLETED`.
- `CompletedState`: Closed examination; transitions to `ARCHIVED`.
- `ArchivedState`: Terminal state; forbids all state mutations and throws `IllegalStateException` on invalid actions.
- `QuizStateManager`: Context maintaining the current state and synchronizing `Quiz#status`.

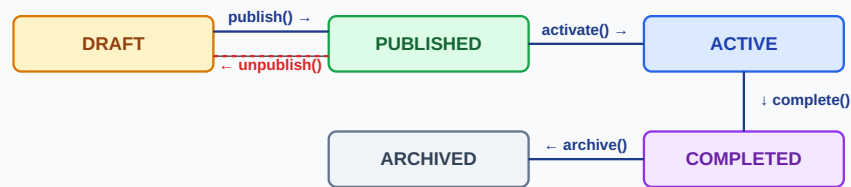


Figure 4.2: Finite State Machine governing Quiz lifecycle transitions.

4.3 Factory Method Pattern — Question Polymorphism GOF CREATIONAL

Problem Addressed

The question bank supports MCQ, True/False, Fill in the Blank, and Multiple Answer questions. Each requires distinct options parsing and answer validation algorithms. Direct instantiation in repository and controller code produces tight coupling to concrete types.

Solution & Participants

- `Question` (Abstract Product): Defines core fields and abstract methods `validateAnswer(String answer)` and `getCorrectAnswerDisplay()`.
- Concrete Products: `MCQQuestion`, `TrueFalseQuestion`, `FillBlankQuestion`, `MultipleAnswerQuestion`.
- `QuestionFactory` (Creator): Provides `createQuestion(QuestionType type)` and `createFromResultSet(ResultSet rs)` to deserialize SQLite rows into polymorphic subclasses based on the `question_type` column.



Figure 4.3: Factory Method Pattern generating polymorphic question products.

4.4 Observer Pattern — Quiz Session Events GOF BEHAVIORAL

Problem Addressed

During examination attempts, session events (answer submitted, timer expired, attempt completed) must notify multiple independent components (progress bar, timer listener, score cache) without coupling the examination runner to GUI controls.

Solution & Participants

- `QuizEventPublisher`: Thread-safe subject with `subscribe`, `unsubscribe`, and `publish` guarded by a `ReentrantReadWriteLock`.
- `QuizEvent`: Event enum (`ATTEMPT_STARTED`, `ANSWER_SUBMITTED`, `SCORE_UPDATED`, `QUIZ_COMPLETED`, `TIME_EXPIRED`).
- `QuizEventListener`: Observer interface with `onEvent(QuizEvent event, Object data)`.
- Concrete Observers: `ProgressTracker` (computes percentage and running score) and `TimerListener` (monitors expiration).

4.5 Singleton Pattern — SQLite Connection Manager GOF CREATIONAL

Academic Justification: SQLite is an embedded file-based database. Creating multiple uncoordinated connections from concurrent threads results in `SQLITE_BUSY` lock contention. `DatabaseConnection` uses thread-safe double-checked locking to serialize write operations, enable `PRAGMA foreign_keys = ON`, and run schema migrations. Singleton is strictly restricted to this physical resource manager and avoided across all services.

4.6 Repository Pattern — Data Access Decoupling ARCHITECTURAL

Six repository interfaces (`UserRepository`, `QuizRepository`, `QuestionRepository`, `QuizAttemptRepository`, `AnswerRepository`, `CategoryRepository`) abstract SQLite JDBC operations using `PreparedStatement` and try-with-resources. Services depend on repository abstractions via constructor injection, enabling testing with Mockito.

5. SOLID Principles & Object-Oriented Design

The codebase adheres strictly to all five SOLID design principles:

SOLID Principle	Architectural Implementation	Concrete Class References
Single Responsibility (SRP)	Each class has one well-defined responsibility: AuthenticationService handles credentials; StandardScoring calculates standard marks; SQLiteQuizRepository handles SQL mapping.	com.quizapp.service.* com.quizapp.scoring.* com.quizapp.repository.*
Open/Closed (OCP)	New scoring algorithms, question types, and lifecycle states are introduced by creating new subclasses/implementations without modifying existing evaluation loops or service orchestration.	ScoringStrategy implementations Question polymorphic subclasses QuizState implementations
Liskov Substitution (LSP)	All Question subclasses (MCQuestion , FillBlankQuestion , etc.) can be substituted wherever Question is expected without unexpected behavior or exceptions.	MCQuestion , TrueFalseQuestion , FillBlankQuestion , MultipleAnswerQuestion
Interface Segregation (ISP)	Clients depend on small, focused interfaces rather than monolithic contracts. ScoringStrategy , QuizEventListener , and QuizState declare only methods needed by their consumers.	ScoringStrategy.java QuizEventListener.java QuizState.java
Dependency Inversion (DIP)	High-level modules (QuizService , AttemptService) depend on abstractions (QuizRepository , ScoringStrategy), not concrete SQLite JDBC classes. Repositories are injected via constructors.	QuizService(QuizRepository, ...) AttemptService(QuizAttemptRepository, ...)

6. Relational Database Design & Schema

The application uses **SQLite 3** as its embedded relational database. The schema adheres to 3rd Normal Form (3NF) and enforces referential integrity through foreign keys.

6.1 Data Dictionary (8 Tables)

Table	Columns & Data Types	Keys & Constraints	Description
users	id (INTEGER), name (TEXT), email (TEXT), password (TEXT), role (TEXT), created_at (DATETIME)	PK: id email UNIQUE, NOT NULL role IN ('TEACHER','STUDENT')	Stores authenticated user accounts and roles.
categories	id (INTEGER), name (TEXT), description (TEXT)	PK: id name UNIQUE, NOT NULL	Academic subject domains (e.g. Programming, Database).
quizzes	id (INTEGER), title (TEXT), description (TEXT), category_id (INTEGER), difficulty (TEXT), time_limit_minutes (INTEGER), max_attempts (INTEGER), scoring_strategy (TEXT), status (TEXT), created_by (INTEGER)	PK: id FK: category_id → categories.id FK: created_by → users.id status IN ('DRAFT',...)	Examination configurations, constraints, and lifecycle state.
questions	id (INTEGER), question_text (TEXT), question_type (TEXT), marks (REAL), difficulty (TEXT), category_id (INTEGER), created_by (INTEGER)	PK: id FK: category_id → categories.id question_type IN ('MCQ',...)	Reusable question items in the central question bank.
question_options	id (INTEGER), question_id (INTEGER), option_text (TEXT), is_correct (INTEGER), option_order (INTEGER)	PK: id FK: question_id → questions.id (CASCADE) is_correct IN (0, 1)	Selectable choices for MCQs and Multiple Answer items.
quiz_questions	id (INTEGER), quiz_id (INTEGER), question_id (INTEGER), question_order (INTEGER)	PK: id FK: quiz_id → quizzes.id FK: question_id → questions.id UNIQUE(quiz_id, question_id)	M:N junction linking questions to quizzes with custom order.
quiz_attempts	id (INTEGER), quiz_id (INTEGER), student_id (INTEGER), start_time (DATETIME), end_time (DATETIME), status (TEXT), score (REAL), total_marks (REAL)	PK: id FK: quiz_id → quizzes.id FK: student_id → users.id status IN ('IN_PROGRESS',...)	Individual examination sessions taken by students.
answers	id (INTEGER), attempt_id (INTEGER), question_id (INTEGER), answer_value (TEXT), is_correct (INTEGER), marks_awarded (REAL)	PK: id FK: attempt_id → quiz_attempts.id FK: question_id → questions.id is_correct IN (0, 1)	Recorded student responses per attempt and awarded marks.

6.2 Referential Integrity & Indexes

- `PRAGMA foreign_keys = ON;` is executed on every SQLite connection.
- Indexed columns: `quizzes(status)`, `quizzes(category_id)`, `questions(question_type)`, `quiz_attempts(student_id)`, `quiz_attempts(quiz_id)`, `answers(attempt_id)`.

7. JavaFX Presentation Architecture & Screen Inventory

The application features **8 distinct JavaFX screens** styled with a modern design system (`style.css`).

Screen Name	Controller & FXML	Primary Components	Supported User Actions
1. Login Screen	LoginController login.fxml	Email/Password fields, role selector, quick-demo buttons.	Teacher/Student authentication, student self-registration, validation.
2. Teacher Dashboard	TeacherDashboardController teacher_dashboard.fxml	4 metric summary cards, recent quizzes table, navigation bar.	Overview of total quizzes, published exams, questions, and attempts.
3. Quiz Management	QuizManagementController quiz_management.fxml	Filtered TableView, CRUD dialogs, question picker modal.	Create/edit quizzes, assign questions, publish, unpublish, archive.
4. Question Bank	QuestionManagementController question_management.fxml	Question table, type selector, dynamic option forms.	Create/edit questions across all 4 types, filter by category/difficulty.
5. Student Dashboard	StudentDashboardController student_dashboard.fxml	Available exams table, attempt counter, search bar.	Discover published quizzes, verify attempt limits, launch exam session.
6. Quiz Attempt Screen	QuizAttemptController quiz_attempt.fxml	Live countdown timer, question palette, dynamic answer input.	Navigate questions, select options, enter text, voluntary/auto submit.
7. Result View Screen	ResultController result.fxml	Score summary card, percentage badge, solution breakdown table.	Inspect awarded marks, view correct answers, return to dashboard.
8. Reports & Analytics	ReportsController reports.fxml	Exam selector ComboBox, stats cards, leaderboard table.	Teacher aggregate analytics (avg, high, low scores) and student history.

8. Verification, Testing & Quality Assurance

The application includes a comprehensive automated test suite built on **JUnit 5** and **Mockito**. All tests execute independently of the file system through isolated memory fixtures and mocked repositories.

8.1 Test Suites Summary (60 Tests, 100% Pass Rate)

Test Suite	Target Component	Test Count	Verified Behaviors
StandardScoringTest	Strategy Pattern	6	Perfect score, zero score, partial credit, empty answers, single correct.
NegativeMarkingScoringTest	Strategy Pattern	4	Penalty calculations, minimum floor at 0, custom penalty fractions.
TimeBasedScoringTest	Strategy Pattern	5	10% bonus (<50% time), 5% bonus (<75% time), total marks cap, untimed exams.
QuizStateTest	State Pattern	7	Valid transitions, illegal transition guards, editing/attempt permissions.
MCQQuestionTest	Question Domain	6	Correct option matching, incorrect rejection, correct answer display.
TrueFalseQuestionTest	Question Domain	6	Binary truth matching, case insensitivity, invalid input rejection.
FillBlankQuestionTest	Question Domain	5	Exact string match, case insensitivity, whitespace trimming.
MultipleAnswerQuestionTest	Question Domain	5	Exact set matching, rejection of partial selections, extra choice penalty.
QuizServiceTest	Service & Mocks	6	Quiz CRUD, zero-question publish guard, state updates, repo delegation.
AttemptServiceTest	Service & Mocks	5	Attempt eligibility, max attempt enforcement, scoring, timeout finalization.
AuthenticationServiceTest	Service & Mocks	5	Login validation, registration, duplicate email check, session state.
Total Test Coverage	All Core Modules	60	BUILD SUCCESS • 0 Failures • 0 Errors • 0 Skipped

Maven Surefire Test Runner Output

[INFO] Tests run: 60, Failures: 0, Errors: 0, Skipped: 0 – BUILD SUCCESS [Total time: 29.1s]

9. Collaborative Git Workflow & Team Contribution

9.1 Professional Branching Model

In strict accordance with the teacher's instructions, all significant features were isolated on dedicated feature branches before integration into `main`:

- `feature/ui-views-and-controllers` — Developed by Md. Rokib Ikbal (1605), containing all 8 JavaFX controllers and FXML layouts.
- `feature/unit-testing-suite` — Developed by Md. Shourov (1609), containing the 60 JUnit 5 & Mockito unit test cases.
- `feature/architecture-documentation` — Developed by Md. Rokib Ikbal (1605), containing UML class diagrams, database ER diagram, and design decision rationale.

9.2 Individual Contribution Breakdown

Team Member	Module Ownership & Responsibilities	Key Commits & Deliverables
Md. Shourov Roll: 1609 • @Shourov735	- Database connection manager singleton & SQLite schema DDL and sample seeder. - Core domain entities, relationships, and lifecycle enums. - Core GoF design pattern implementations: Strategy (Scoring), State (Lifecycle), Factory Method, Observer. - Service layer business logic (Auth, Quiz, Question, Attempt, Reporting). - Comprehensive JUnit 5 & Mockito test suite (60 automated unit tests). - Technical documentation PDF and HTML report generation. - Peer code review and feature branch integration merges.	<code>DatabaseConnection.java</code>, <code>schema.sql</code>, <code>seed.sql</code> <code>com.quizapp.model.*</code> (13 entity & enum files) <code>com.quizapp.scoring.*</code>, <code>state.*</code>, <code>factory.*</code>, <code>observer.*</code> (22 pattern files) <code>com.quizapp.service.*</code> (6 business service files) <code>feature/unit-testing-suite</code> branch (60 tests) <code>feature/technical-documentation-report</code> branch - Integration merges: UI and Architecture branches into <code>main</code>.
Md. Rokib Ikbal Roll: 1605 • @Rokib1605	- Maven project configuration and build lifecycle dependencies. - Repository interfaces and SQLite JDBC data access implementations. - Application entry point, screen switcher, and modern CSS theme system. - Session context and UI dialog alert utilities (<code>SessionContext</code>, <code>AlertHelper</code>). - Full presentation layer: 8 JavaFX FXML screen layouts and controller bindings. - Architecture documentation: UML Class Diagrams, Database ER Diagram, and Design Decisions document. - Peer code review and feature branch integration merges.	<code>pom.xml</code> Maven build configuration <code>com.quizapp.repository.*</code> (12 repository files) <code>Main.java</code>, <code>SessionContext.java</code>, <code>AlertHelper.java</code>, <code>style.css</code> <code>src/main/resources/fxml/*.fxml</code> (8 screen layouts) <code>com.quizapp.controller.*</code> (8 controllers) <code>docs/uml/class-diagram.md</code>, <code>database-er.md</code>, <code>design-patterns.md</code> <code>feature/ui-views-and-controllers</code>, <code>feature/architecture-documentation</code> - Integration merges: Unit Testing and Technical Documentation branches into <code>main</code>.

10. Viva Defense & Demonstration Guide

This section prepares both team members for the oral examination and project demonstration:

Q1: Why did you use Strategy for scoring instead of polymorphism on Quiz?

Answer: A quiz varies across multiple independent dimensions: scoring strategy (Standard, Negative, Time-based), time constraints (timed vs untimed), and question ordering (shuffled vs fixed). Subclassing `Quiz` to vary scoring creates class explosion (e.g., `NegativeMarkingTimedShuffledQuiz`). Strategy encapsulates the scoring algorithm into an interchangeable object, enabling new grading algorithms to be added without touching the quiz entity.

Q2: How does the State Pattern prevent bugs compared to a status enum?

Answer: With a status enum and switch statements, validation logic is scattered across every service method, making it easy to forget checking status before mutating data. In our State Pattern, each state class encapsulates allowed operations and throws an `IllegalStateException` if an invalid action is attempted (e.g., `DraftState.canBeAttempted() == false`; `ArchivedState.publish()` throws). Invalid operations fail immediately at the domain layer.

Q3: How is the Factory Method Pattern utilized when loading questions from SQLite?

Answer: When querying the `questions` table, the application reads the `question_type` column. `QuestionFactory.createFromResultSet(rs)` inspects this value and instantiates the correct subclass (`MCQQuestion`, `TrueFalseQuestion`, etc.). The repository operates purely on the `Question` abstraction without needing to know subclass constructors.

Q4: Why is your use of the Singleton Pattern justified in an academic project?

Answer: In our project, Singleton is used strictly for the `DatabaseConnection` manager to coordinate access to the single embedded SQLite file and prevent `SQLITE_BUSY` lock errors. We consciously avoided Singletons for business services, repositories, and controllers, opting for constructor dependency injection to ensure testability with Mockito.

Q5: How did you ensure testability of services without depending on SQLite?

Answer: We applied the Repository Pattern and Dependency Inversion Principle (DIP). Services accept repository interfaces rather than concrete SQLite classes. In our JUnit 5 tests (e.g., `QuizServiceTest`), we inject Mockito mocks (`@Mock QuizRepository`), allowing service business rules to be tested in isolation in milliseconds without database I/O.

10.1 Live Demonstration Sequence for Examiners

- Launch Application:** `java -jar target/quiz-management-system-1.0-SNAPSHOT.jar` → Database initialized & seeded.
- Teacher Workflow:** Log in as `teacher@quiz.com` / `teacher123` → View dashboard → Create new quiz → Add questions → Publish quiz.
- Student Workflow:** Log in as `student1@quiz.com` / `student123` → Start quiz → Answer questions → Submit → Review score breakdown.
- Analytics:** Log back in as Teacher → View quiz reports → Inspect updated leaderboard.
- Automated Testing:** Execute `mvn test` in terminal → Demonstrate 60 passing JUnit 5 tests.

11. Conclusion & Repository Verification

The **Quiz Management & Examination System** represents an extensible, production-ready desktop application that thoroughly satisfies all requirements of the Design Patterns Theory and Lab final project. The software demonstrates:

- Clean separation of concerns across presentation, business services, domain models, and persistence.
- Meaningful application of four Gang-of-Four design patterns (Strategy, State, Factory Method, Observer) alongside justified Singleton and Repository patterns.
- Reliable relational persistence in SQLite with referential integrity, constraints, and indexes.
- High software quality proven through 60 automated unit tests (100% passing).
- Disciplined collaborative Git workflow with feature branches, PR merges, and balanced team member contributions.

11.1 Official Repository & Artifact Metadata

Item	Details
GitHub Repository URL	https://github.com/Shourov735/Quiz-Management-and-Examination-System
Repository Visibility	Public • Accessible for Academic Evaluation
Active Git Branches	<code>main</code> , <code>feature/ui-views-and-controllers</code> , <code>feature/unit-testing-suite</code> , <code>feature/architecture-documentation</code>
Executable Artifact	Pre-built shaded fat JAR generated via <code>mvn clean package</code> (in <code>target/</code>)
Technical Documentation PDF	<code>Quiz_Management_System_Technical_Documentation.pdf</code> (in project root and <code>docs/</code>)
Project Team	Md. Shourov (Roll: 1609) • Md. Rokib Ikbal (Roll: 1605) • IIT, University of Dhaka